`nodegen` et `nodeview`

Tutoriel : génération de nodeFiles pour L-HYPHEN,
algorithmes et améliorations possibles

Répertoire `nodeGen/`

16 octobre 2026

Résumé

`nodegen` produit le fichier de nœuds (*nodeFile*) lu par la commande `readNodeFile` de L-HYPHEN : un pavage de cellules polygonales d'un rectangle, éventuellement entaillé par une pré-fissure rectiligne. Il est conçu pour éviter les deux défauts rencontrés avec les maillages de `cellPrepro` : des barres extrêmement courtes, qui imposent un pas de temps minuscule en flexion, et une pré-fissure obtenue en retirant des cellules. À partir d'un fichier d'input existant pris comme modèle, il produit aussi un `input.txt` complet, directement exécutable par `run`. `nodeview` produit une vue SVG d'analyse de n'importe quel nodeFile. Ce document présente leur utilisation pas à pas, détaille les algorithmes et liste les améliorations envisageables.

Table des matières

1 Premiers pas **2**
- 1.1 Compilation . . . 2
- 1.2 Générer un premier échantillon . . . 2

2 Paramètres **3**

3 Visualiser et contrôler un nodeFile avec `nodeview` **6**

4 Produire le fichier d'input l-hyphen **8**
- 4.1 Input complet à partir d'un modèle (`inputTemplate`) . . . 8
- 4.2 Lignes d'input seules . . . 9
- 4.3 Mors : rangées de nœuds attrapées . . . 9
- 4.4 Vérifications . . . 10

5 Algorithmes **10**
- 5.1 Germes . . . 11
- 5.2 Régularisation de Lloyd . . . 11
- 5.3 Pavage de Voronoï par découpe de demi-plans . . . 11
- 5.4 Sommets partagés et types de sommets . . . 12
- 5.5 Arêtes courtes et jonctions à 3 branches . . . 12
- 5.6 Pré-fissure par symétrie . . . 13
- 5.7 Pointe de fissure . . . 14
- 5.8 Décalage des parois . . . 15

6 Validation sur un essai d'ouverture **15**

7 Limites et améliorations possibles **16**
7.1 Anisotropie du réseau hexagonal 16
7.2 Contrôle du nombre de côtés 17
7.3 Géométrie 17
7.4 Robustesse et outillage 17

1 Premiers pas

1.1 Compilation

Les deux outils sont écrits en C++17 sans aucune dépendance :

```
cd nodeGen
make                 # produit nodegen et nodeview
```

1.2 Générer un premier échantillon

Le fichier `doc/tutorial_params.txt` décrit un échantillon de même géométrie que le cas `Frank-ouverture-fissure` : 13,6 mm × 16 mm, environ 2200 cellules de 0,353 mm de diamètre équivalent, et une pré-fissure de 1,2 mm partant du milieu du bord gauche.

```
# Paramètres du tutoriel : échantillon du type Frank-ouverture-fissure (13.6 mm x 16 mm
    , ~2200 cellules)

# Domaine
xmin 0.0032
ymin 0.002
Lx   0.0136
Ly   0.016

# Cellules
cellSize        3.53e-4   # diamètre équivalent visé
barWidth        2e-6      # écart entre parois voisines (= barWidth de readNodeFile)
lattice         hex       # réseau hexagonal perturbé
disorder        0.3       # perturbation des germes (fraction du pas du réseau)
lloydIterations 5         # régularisation de Lloyd
minEdgeFactor   0.25      # seuil des arêtes courtes (fraction de lRef = sqrt(Lx Ly /
    Ncell))
minSides        5         # transition T1 interdite si une cellule passait sous 5 côtés
     (étirement à la place)
seed            1

# Pré-fissure depuis le bord gauche, à mi-hauteur, longueur 1.2 mm
crack        0.0032 0.010  0.0044 0.010
tipInterface 1             # interface collée prolongeant la fissure au-delà de la
    pointe (en cellules)
opening      2e-6          # écart supplémentaire entre les lèvres (> distGlue)
distGlue     2e-7          # valeur de distGcGlue dans l'input

# Lignes d'input l-hyphen (valeurs de Frank-ouverture-fissure)
cellProperties  2000.0  7.928638584420448e-4  79286385.84420447  0   # Kn Kr Mz_max
    p_int
glueProperties  26373.626373626375  26373.626373626375  1.2575114788066131e-06
gripRows        1             # mors : une rangée de nœuds (celle du bord)
pullVelocity    3e-4          # vitesse imposée aux mors
nodeMass        8e-8          # masse des nœuds (pas de temps critique)
```

```
# Sorties
output work/nodefile.txt
svg    work/nodefile.svg
input  work/nodegen_input.txt

# Input complet : copie de modele_input.txt (input de Frank) avec les lignes du
    maillage remplacées
inputTemplate  modele_input.txt  work/input.txt
```

```
cd nodeGen/doc
../nodegen tutorial_params.txt
```

La génération prend moins d'un dixième de seconde. `nodegen` écrit le nodeFile (`output`), une vue SVG d'analyse (`svg`), les lignes d'input L-HYPHEN propres au maillage (`input`) et, ici, un fichier d'input complet construit à partir de `modele_input.txt` (`inputTemplate`, section 4) :

```
cd work && ../../../run input.txt     # le calcul se lance directement
```

`nodegen` affiche un bilan :

```
nodegen
  domaine           : [0.0032, 0.0168] x [0.002, 0.018]
  cellules          : 2222 (visé 2223), noeuds : 13150
  arêtes courtes    : 67 sous le seuil 0.25 x lRef = 7.823e-05 (lRef = sqrt(Lx Ly / 2222) = 0.0003129)
                      32 T1 (longueur 9.388e-05), 35 étirées, 0 amas contractés puis éclatés
  sommets éclatés   : 0 (4 branches ou plus)
  branches/sommet   : 2: 4  3: 4434  4: 4  (4 sur la fissure ouverte, extrémités comprises)
  barres            : l_moy = 0.0001975, l_min = 7.693e-05 (l_min/l_moy = 0.3895, l_min/lRef = 0.2458)
  cellules par nombre de côtés (intérieures : sans sommet sur le bord du domaine) :
      côtés               toutes         intérieures
          4         28 (  1.3 %)          2 (  0.1 %)
          5        188 (  8.5 %)         88 (  4.3 %)
          6       1945 ( 87.5 %)       1903 ( 92.8 %)
          7         60 (  2.7 %)         56 (  2.7 %)
          8          1 (  0.0 %)          1 (  0.0 %)
      total       2222                 2050
  parois            : 12966 collées, 184 libres (bords + fissure)
  pré-fissure       : 8 parois ouvertes (barWidth + 2e-06), pointe effective en (0.004381, 0.01) au lieu de
     (0.0044, 0.01)
  fichiers          : work/nodefile.txt, work/nodefile.svg, work/nodegen_input.txt
  input complet     : work/input.txt (modèle modele_input.txt, fragments dans work/nodegen_input.txt)
  mors bas          : 82 noeuds (1 rangée)
  mors haut         : 80 noeuds (1 rangée)
                      dt_crit(flexion) = 7.728e-07
```

Les lignes à surveiller :

- **arêtes courtes** : seuil ℓ_c et longueur caractéristique ℓ_{ref} utilisés, nombre d'arêtes sous le seuil et la façon dont elles ont été traitées (transitions T1, étirements, amas, section 5.5) ;
- **branches/sommet** : nombre de sommets par nombre de branches ; seuls les sommets de la fissure ouverte (lèvres et extrémités) peuvent en avoir 4 ;
- **barres** : le rapport $\ell_{\min}/\ell_{\text{ref}}$ doit être de l'ordre de `minEdgeFactor` (un peu moins, à cause du décalage des parois) ; c'est $\ell_{\min}$ qui fixe le pas de temps critique de flexion dans L-HYPHEN ($\Delta t_{\text{crit}} \propto \ell_{\min}$) ;
- **cellules par nombre de côtés** : effectif et pourcentage des cellules à n côtés, pour toutes les cellules et pour les cellules intérieures (sans sommet sur le bord du domaine), les cellules de bord étant coupées par le rectangle ;
- **pré-fissure** : position effective de la pointe, qui est toujours un sommet du pavage ;
- **lignes d'input** : nombre de nœuds dans chacun des mors haut et bas.

2 Paramètres

Le fichier de paramètres contient un mot-clé par ligne ; tout ce qui suit `#` est un commentaire.

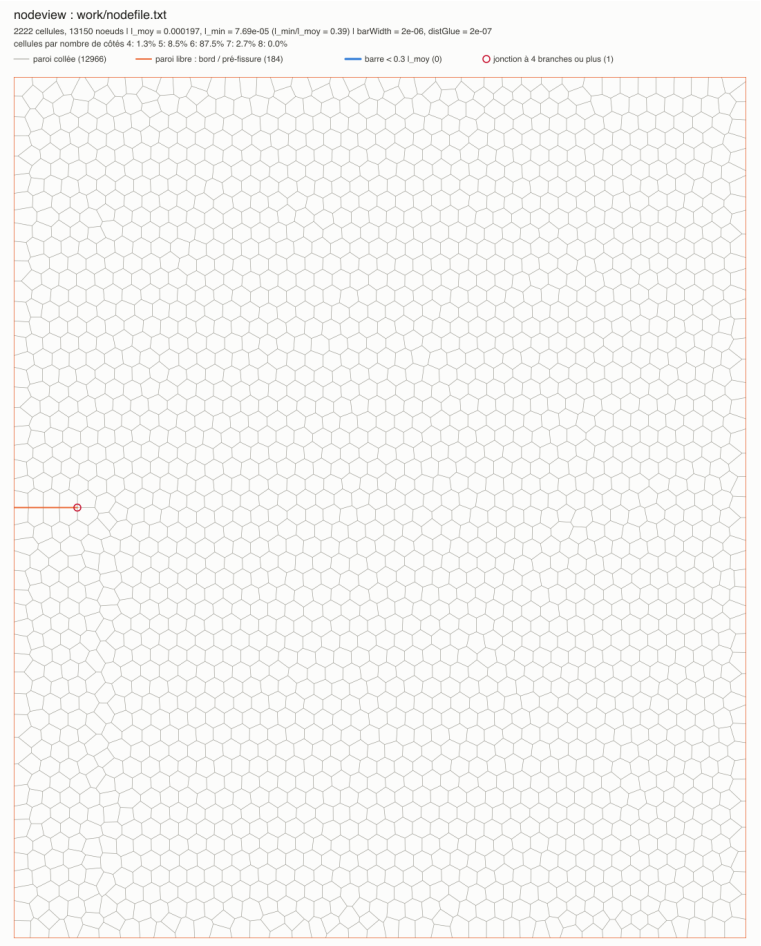
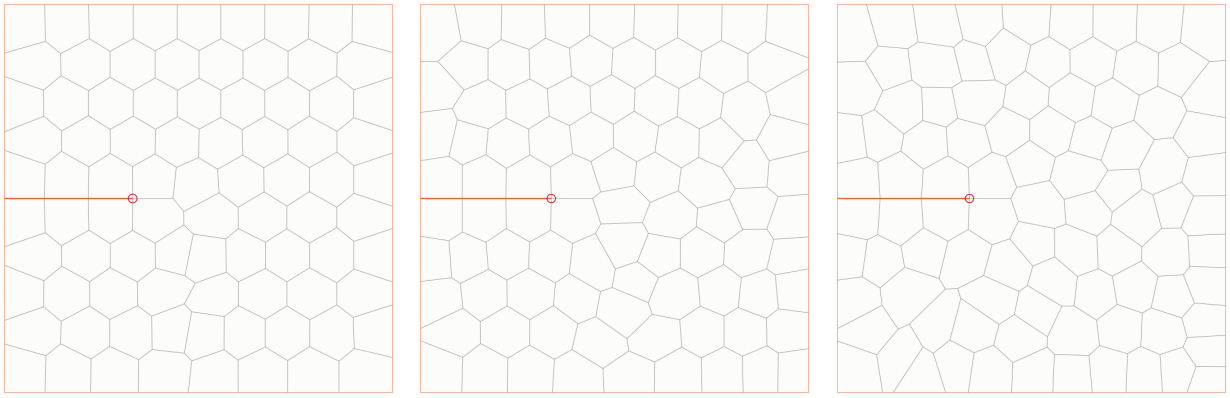


FIGURE 1 – Vue produite par `nodegen` pour `tutorial_params.txt`. En gris : parois collées ; en orange : parois libres (bords du domaine et pré-fissure) ; en bleu : barres courtes (aucune ici) ; cerclée de rouge : jonction à 4 branches (seule la pointe de la fissure).

Mot-clé	Défaut	Rôle
`xmin`, `ymin`	0, 0	coin inférieur gauche du domaine
`Lx`, `Ly`	0.01	dimensions du domaine
`cellSize`	3.5e-4	diamètre équivalent visé ; le nombre de cellules vaut $L_x L_y/(\pi d^2/4)$
`barWidth`	2e-6	largeur des barres, à reporter dans `readNodeFile`
`lattice`	`hex`	`hex` : réseau hexagonal perturbé ; `random` : tirage aléatoire
`disorder`	0.3	perturbation des germes du réseau `hex` (fraction du pas)
`lloydIterations`	10	itérations de régularisation de Lloyd
`seed`	1	graine du générateur aléatoire
`minEdgeFactor`	0.2	seuil des arêtes courtes, en fraction de $\ell_{\text{ref}} = \sqrt{L_x L_y/N}$ (N : nombre de cellules)
`t1Margin`	1.2	longueur de l'arête créée par une transition T1 ou un éclatement, en multiple du seuil
`minSides`	5	une transition T1 ne peut faire passer une cellule sous `minSides` côtés ; sinon l'arête est étirée
`crack x0 y0 x1 y1`	—	pré-fissure (segment), facultative
`opening`	2e-6	écart supplémentaire entre les lèvres de la fissure
`tipInterface`	1	longueur (en cellules) de l'interface collée qui prolonge la fissure
`distGlue`	2e-7	valeur de `distGcGlue` de l'input, pour les contrôles
`cellProperties`	—	`Kn Kr Mz_max p_int` reportés dans `readNodeFile`
`glueProperties`	—	`kn_coh kt_coh Gc` reportés dans `setGcGlueSameProperties`
`gripRows`	1	nombre de rangées de nœuds attrapées par chacun des mors bas et haut
`gripHeight`	—	hauteur explicite des mors (si `gripRows` n'est pas donné)
`pullVelocity`	3e-4	vitesse imposée aux mors (bas $-v$, haut $+v$)
`nodeMass`	—	masse des nœuds, pour estimer le pas de temps critique de flexion
`inputTemplate`	—	`modèle [sortie]` : input complet produit à partir d'un input existant (sortie par défaut `input.txt`)
`output`, `svg`, `input`		nodeFile, vue SVG, lignes d'input L-HYPHEN

Choisir la régularité. `lattice` et `disorder` règlent la forme des cellules (figure 2). Sur la géométrie du tutoriel, avec `lloydIterations 5` et `minSides 5`, on obtient pour les cellules intérieures :

Réglage	4 côtés	5 côtés	6 côtés	7 côtés
`lattice random`, `lloydIterations 10`	0 %	23 %	54 %	22 %
`lattice hex`, `disorder 0.2`	0,1 %	3,0 %	95 %	1,5 %
`lattice hex`, `disorder 0.3`	0,1 %	4,3 %	93 %	2,7 %
`lattice hex`, `disorder 0.4`	0 %	10 %	81 %	8,5 %



(a) `disorder 0` (b) `disorder 0.3` (c) `disorder 0.5`

FIGURE 2 – Effet de `disorder` (réseau hexagonal, domaine de 3 mm, pré-fissure de 1 mm). Le long de la fissure, la rangée de cellules est symétrique ; plus loin, le réseau est régulier.

Les transitions T1 qui éliminent les arêtes courtes (section 5.5) retirent un côté à deux cellules et en ajoutent un à deux autres : pentagones et heptagones apparaissent par paires, d'où des proportions voisines.

Pré-fissure. Le segment `crack` peut partir d'un bord (fissure débouchante) ou être entièrement intérieur, et avoir une orientation quelconque. `opening` doit être nettement supérieur à `distGlue` pour que les lèvres ne soient pas collées. `tipInterface` fixe l'étendue de la symétrie au-delà de la pointe, donc la distance entre la pointe et le défaut de raccordement au réseau (section 5.7).

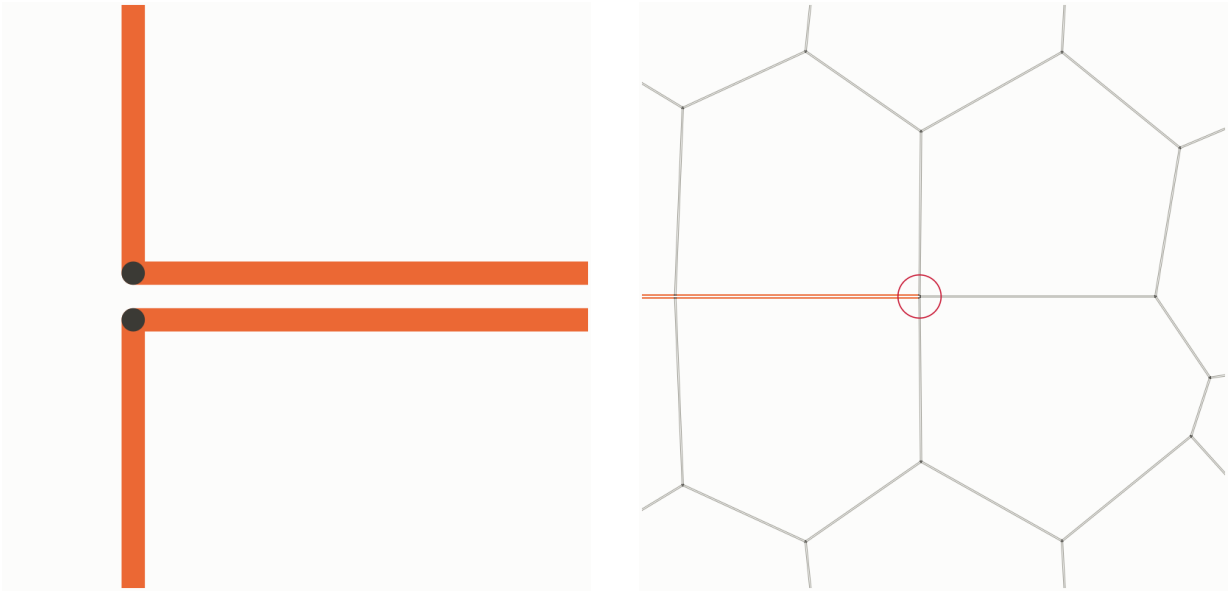
3 Visualiser et contrôler un nodeFile avec `nodeview`

```
nodeview nodefile.txt [-w barWidth] [-g distGlue] [-r ratio] [-box x0 x1 y0 y1]
                      [-nodes] [-bare] [-px largeur] [-o out.svg]
```

`nodeview` lit n'importe quel nodeFile (y compris ceux de `cellPrepro`) comme `readNodeFile` le fait, et produit un SVG (vectoriel : le zoom est illimité dans un navigateur). Chaque paroi est classée :

- **collée** si son milieu est à moins de `barWidth + distGlue` d'une barre d'une autre cellule — c'est le critère de création des liens cohésifs de L-HYPHEN, et le nombre de parois collées est égal au nombre de « liens collés » affiché par `diagnostic.txt` ;
- **libre** sinon : bord du domaine, pré-fissure, trou ;
- les barres plus courtes que `ratio` $\times\,\bar{\ell}$ (défaut 0.3) sont surlignées en **bleu**. Dans la vue produite par `nodegen`, ce sont les barres dont l'arête du pavage est sous le seuil ℓ_c (section 5.5), c'est-à-dire exactement les arêtes signalées en erreur par le contrôle final ; le décalage des parois raccourcit chaque barre d'environ `barWidth`, et une arête étirée au seuil n'est donc pas signalée ;
- les jonctions à 4 branches ou plus sont cerclées de **rouge** : au moins 4 cellules en un sommet, ou 3 cellules sur un bord libre (2 parois + le bord). Les jonctions sont reconstruites à partir des parois collées : chaque extrémité d'une paroi collée est rattachée au nœud le plus proche de la cellule voisine, si bien que les deux lèvres libres d'une fissure ne sont pas rattachées entre elles.

`nodeview` affiche aussi le nombre de jonctions par nombre de cellules. Sur un nodeFile de `nodegen`, seules les extrémités de la pré-fissure sont cerclées. Sans `-w`, la largeur des barres est estimée par la distance minimale entre nœuds de cellules différentes. `-box` restreint le dessin à une fenêtre (zoom), `-nodes` dessine les nœuds, `-bare` supprime titre et légende.



(a) Embouchure de la fissure (fenêtre de 50 µm) (b) Pointe de la fissure (fenêtre de 0,8 mm)

FIGURE 3 – Zooms obtenus avec `-box` et `-nodes`. (a) Les lèvres de la fissure sont écartées de `barWidth + opening` = 4 µm, les parois collées de `barWidth` = 2 µm. (b) La fissure se termine sur un sommet à 4 branches (le seul du maillage) et se prolonge par une interface collée, qui part en zigzag.

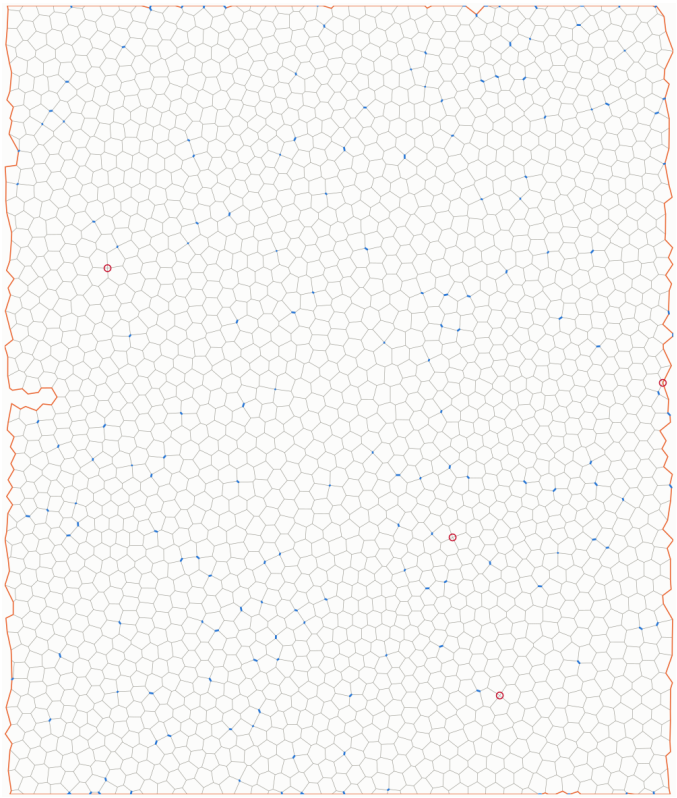


FIGURE 4 – `nodeview` appliqué au nodeFile de `Frank-ouverture-fissure` (produit par `cellPrepro`) : pré-fissure obtenue en retirant des cellules, bords latéraux irréguliers et 331 barres plus courtes que $0.3\,\bar{\ell}$ (en bleu), la plus courte mesurant $0.012\,\bar{\ell}$.

4 Produire le fichier d'input l-hyphen

4.1 Input complet à partir d'un modèle (`inputTemplate`)

Le plus simple est de partir d'un fichier d'input existant, par exemple celui d'une étude précédente :

```
inputTemplate  modele_input.txt  work/input.txt
```

`nodegen` recopie le modèle dans `work/input.txt` en remplaçant les lignes qui dépendent du maillage, marquées `# [nodegen]`. Tous les autres réglages du modèle (pas de temps, raideurs de contact, masses, amortissements, sorties…) sont conservés, y compris les constantes `define` et les expressions.

Ligne du modèle	Traitement
`readNodeFile`	nouveau nodeFile (chemin relatif au répertoire de l'input produit) et `barWidth` ; `Kn Kr Mz_max p_int` de `cellProperties`, sinon recopiés du modèle
`cleanShortBars`	commentée (inutile)
2 `setNodeControlInBox`	boîtes recalculées sur le nouveau domaine, de hauteur fixée par `gripRows` (section 4.3) ; modes et valeurs repris du modèle, sauf si `pullVelocity` est donné
2 `captureNodes`	boîtes recalculées, noms de fichiers conservés
`setGcGlueSameProperties`	remplacée si `glueProperties` est donné, sinon recopiée
`distGcGlue`, `glue`	recopiée ; `nodegen` vérifie qu'elle reste inférieure à `opening`
autres lignes	recopiées telles quelles

Les mors bas et haut sont reconnus comme les deux boîtes de plus petit et de plus grand $y_{\min}$. Si le modèle ne contient pas exactement deux `setNodeControlInBox` (ou deux `captureNodes`), ces lignes sont recopiées sans modification et un avertissement est affiché. `nodegen` affiche le nombre de nœuds de chaque mors et, si la masse des nœuds et k_r sont connus, le pas de temps critique de flexion ; il refuse d'écraser le modèle.

Différences entre le modèle (input de `Frank-ouverture-fissure`) et l'input produit pour le tutoriel :

```
0a1,2
> # Input généré par nodegen à partir du modèle modele_input.txt et de tutorial_params.txt
> # Les lignes modifiées par nodegen sont marquées [nodegen] ; les autres sont celles du modèle.
27c29
< readNodeFile  nodefile.txt   2e-06        2000.0   0.0007928638584420448   79286385.84420447   0
---
> readNodeFile  nodefile.txt  2e-06  2000  0.000792863858442045  79286385.8442045  0    # [nodegen]
30c32
< cleanShortBars 0.3
---
> # cleanShortBars 0.3    # [nodegen] inutile : l_min/l_moy = 0.39
41,42c43,44
< setNodeControlInBox 0.0032085515974282354       0.01679607384100938     0.002   0.0024009500000000002
      1        0.0       0      -3e-04
< setNodeControlInBox 0.0032085515974282354       0.01679607384100938    0.017599049999999998    0.018
     1        0.0       0      3e-04
---
> setNodeControlInBox  0.0032  0.0168  0.002  0.00203798871599668  1 0.0  0 -0.0003    # [nodegen] mors bas
> setNodeControlInBox  0.0032  0.0168  0.0179605794002941  0.018  1 0.0  0 0.0003    # [nodegen] mors haut
45,46c47,48
< captureNodes      bottom.txt 0.0032085515974282354       0.01679607384100938       0.002
    0.0024009500000000002
< captureNodes      top.txt    0.0032085515974282354       0.01679607384100938       0.017599049999999998
      0.018
---
> captureNodes  bottom.txt  0.0032  0.0168  0.002  0.00203798871599668    # [nodegen]
> captureNodes  top.txt  0.0032  0.0168  0.0179605794002941  0.018    # [nodegen]
53c55
< setGcGlueSameProperties 26373.626373626375 26373.626373626375 1.2575114788066131e-06
```

```
---
> setGcGlueSameProperties  26373.6263736264  26373.6263736264  1.25751147880661e-06    # [nodegen]
```

L'input produit a été lancé sans autre modification que la durée simulée (20 000 pas) : aucune rupture au démarrage et 12902 liens collés, comme attendu.

4.2 Lignes d'input seules

Sans modèle, `nodegen` écrit toujours dans le fichier `input` (défaut `nodegen_input.txt`) les lignes d'input qui dépendent du maillage, à copier dans un fichier d'input aux endroits indiqués :

- `readNodeFile` avec le bon `barWidth` et les propriétés de `cellProperties` ;
- les mors : deux `setNodeControlInBox` sur toute la largeur (section 4.3), libres en x (mode 1, force nulle) et à vitesse imposée $\mp v$ en y (mode 0), avec le nombre de nœuds de chaque mors, et les deux `captureNodes` correspondants ;
- `distGcGlue` et `setGcGlueSameProperties` (propriétés de `glueProperties`) ;
- en commentaire, les statistiques du maillage, la position effective de la pointe et, si `nodeMass` est donné, le pas de temps critique de flexion $\ell_{\min}\sqrt{m/k_r}$.

Une ligne dont une valeur n'est pas connue est écrite en commentaire, avec des repères `<...>` à compléter. Pour `tutorial_params.txt` :

```
# ===== Lignes d'input générées par nodegen (tutorial_params.txt) =====
# Domaine [0.0032, 0.0168] x [0.002, 0.018], 2222 cellules, 13150 noeuds
# Barres : l_moy = 0.0001975, l_min = 7.693e-05 (l_min/l_moy = 0.3895) : cleanShortBars est inutile
# Pré-fissure : de (0.0032, 0.01) à la pointe effective (0.004381, 0.01), lèvres écartées de barWidth +
    opening = 4e-06
# Les lignes commentées contiennent des valeurs <...> à compléter (ou à fournir dans le fichier de
# paramètres de nodegen : cellProperties, glueProperties).

# --- Échantillon (section SAMPLE)
#             fichier        barWidth   Kn   Kr   Mz_max   p_int
readNodeFile  work/nodefile.txt  2e-06  2000  0.000792863858442045  79286385.8442045  0
# setNodeMasses 8e-08  ->  dt_crit(flexion) = l_min * sqrt(m/Kr) = 7.728e-07 s (dt <= 7.728e-08 conseillé)
# (masses, amortissements, cellContent... ici)

# --- Chargement : mors bas et haut, libres en x (mode 1 = force nulle), vitesse imposée en y (mode 0)
#                   xmin  xmax  ymin  ymax  xmode xvalue ymode yvalue
setNodeControlInBox  0.0032  0.0168  0.002  0.00203798871599668  1 0.0  0 -0.0003    # bas  : 82 noeuds (1
    rangée)
setNodeControlInBox  0.0032  0.0168  0.0179605794002941  0.018  1 0.0  0 0.0003    # haut : 80 noeuds (1
    rangée)
captureNodes  bottom.txt  0.0032  0.0168  0.002  0.00203798871599668
captureNodes  top.txt     0.0032  0.0168  0.0179605794002941  0.018

# --- Collage (après les contrôles) : distGcGlue doit rester inférieur à opening = 2e-06
distGcGlue 2e-07
setGcGlueSameProperties  26373.6263736264  26373.6263736264  1.25751147880661e-06
```

L'ordre compte : `readNodeFile` ouvre la section de l'échantillon (avant les masses et les amortissements), les contrôles et `captureNodes` suivent, et le collage vient en dernier, une fois `distVerlet` défini. Le nombre de nœuds de chaque mors permet de vérifier que la hauteur des mors est adaptée ; `nodegen` signale un mors vide ou une pré-fissure qui entrerait dans un mors.

4.3 Mors : rangées de nœuds attrapées

Les mors bas et haut sont deux boîtes `setNodeControlInBox` de toute la largeur du domaine. Par défaut (`gripRows 1`), chacune attrape **une seule rangée de nœuds** : les nœuds du bord, tous situés à `barWidth`/2 du bord du domaine. Les parois des cellules de bord restent alors libres de se déformer ; seuls leurs nœuds sur le bord sont pilotés.

Les rangées sont définies topologiquement : la rangée 1 est formée des sommets du bord, la rangée k des sommets situés à $k-1$ barres du bord (les nœuds des cellules voisines au même sommet comptant pour un seul sommet). Une boîte ne sélectionne que par la hauteur ; `nodegen`

la fait s'arrêter à mi-distance entre le nœud le plus haut des `gripRows` premières rangées et le nœud suivant. Sur l'échantillon du tutoriel :

Mors bas	hauteur de la boîte	nœuds attrapés	dont nœuds des rangées suivantes
`gripRows 1` (défaut)	39 µm	82	0
`gripRows 2`	354 µm	253	45
`gripRows 3`	819 µm	651	303
`gripHeight 4e-4`	400 µm	312	—

La première rangée est attrapée exactement : le nœud suivant est à environ 78 µm du bord. Au-delà, un pavage de Voronoï n'a pas de rangées horizontales (le nœud le plus haut de la rangée 2 est plus haut que le nœud le plus bas de la rangée 3) : la boîte attrape toutes les rangées demandées, plus quelques nœuds bas des rangées suivantes, dont `nodegen` affiche le nombre. `gripHeight` impose directement une hauteur de boîte.

4.4 Vérifications

`cleanShortBars` est inutile avec un nodeFile produit par `nodegen`. Avec les mors d'une rangée, l'input du tutoriel a été lancé sur 20 000 pas : aucune rupture, et des forces égales et opposées sur les deux mors ($-1{,}380$ mN en haut, $1{,}379$ mN en bas à $t = 1$ ms). Après un premier lancement, vérifier dans `diagnostic.txt` que le nombre de « liens collés » est égal au nombre de parois collées de `nodeview`, et que le rapport $\Delta t_{\text{crit}}/\Delta t$ de la ligne « flexion » est confortable. Sur l'échantillon du tutoriel, avec les paramètres mécaniques de `Frank-ouverture-fissure` :

	Frank (`cellPrepro`)	`nodegen` (tutoriel)
cellules / nœuds	2155 / 12798	2222 / 13150
$\ell_{\min}/\bar{\ell}$	0.012	0.39
$\Delta t_{\text{crit,flexion}}$	$2{,}4 \times 10^{-8}$ s	$7{,}7 \times 10^{-7}$ s
liens collés (`diagnostic.txt`)	12542	12966
jonctions à 4 branches (`nodeview`)	4	1 (pointe)
ruptures au démarrage (20 000 pas)	—	0

5 Algorithmes

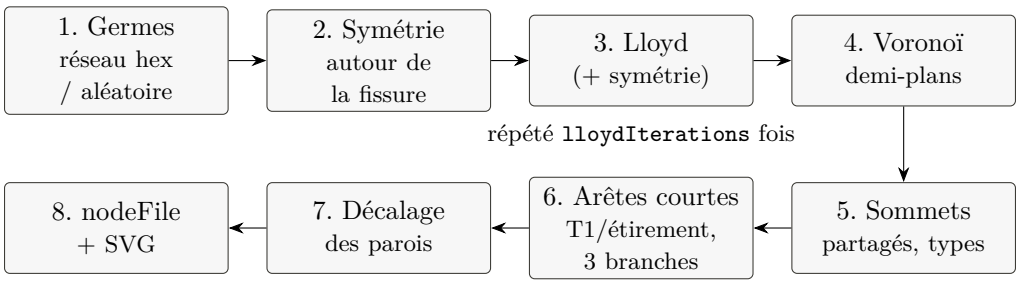
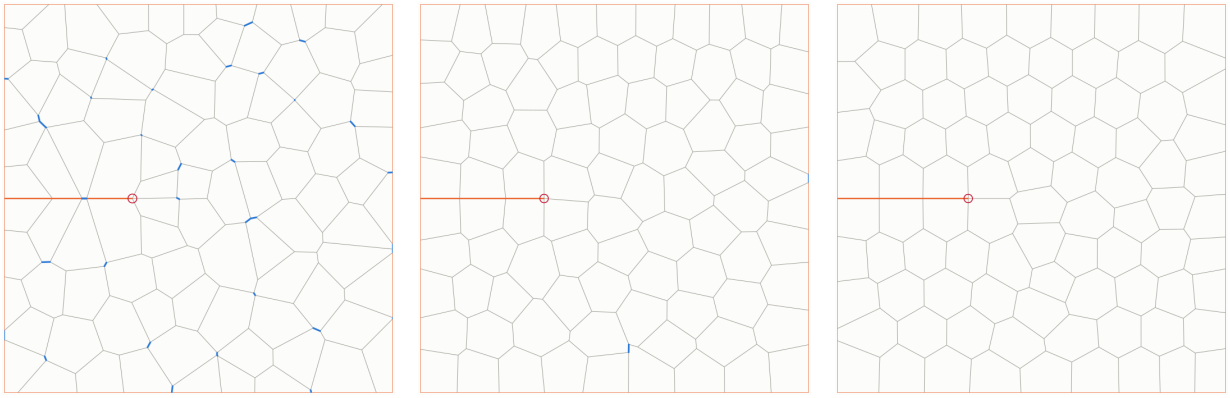


FIGURE 5 – Chaîne de traitement de `nodegen`.



(a) Voronoï brut (b) Lloyd + T1 seule (c) Réseau hex + `minSides 5`

FIGURE 6 – Trois réglages sur un domaine de 3 mm. (a) Germes aléatoires, sans Lloyd ni traitement des arêtes courtes : nombreuses barres courtes (en bleu, plus courtes que $0.3\,\bar{\ell}$), $\ell_{\min} = 0.007\,\ell_{\text{ref}}$ et 29 % d'heptagones. (b) Lloyd (10 itérations), `minEdgeFactor 0.19` et T1 sans limite (`minSides 3`) : $\ell_{\min} = 0.19\,\ell_{\text{ref}}$, 29 % de pentagones, 53 % d'hexagones et 18 % d'heptagones (cellules intérieures). (c) Réseau hexagonal, `disorder 0.3`, `minEdgeFactor 0.25`, T1 limitée par `minSides 5` et étirement : $\ell_{\min} = 0.25\,\ell_{\text{ref}}$, 70 % d'hexagones. Dans les trois cas, tous les sommets ont 3 branches, sauf la pointe de la fissure (cerclée de rouge).

5.1 Germes

Tirage aléatoire (`lattice random`). On vise $N = L_x L_y / A$ cellules d'aire $A = \pi d^2/4$ ($d =$ `cellSize`), d'espacement moyen $s = \sqrt{A}$. Les germes sont tirés uniformément et acceptés s'ils sont à plus de $r_{\min} = 0.7\,s$ des germes déjà placés (recherche par grille) ; si la saturation est atteinte avant N, le complément est tiré sans contrainte.

Réseau hexagonal (`lattice hex`). Un réseau triangulaire de germes, de pas $a = \sqrt{2A/\sqrt{3}}$ (aire de la cellule hexagonale égale à A) et d'interrangée $a\sqrt{3}/2$, donne un pavage d'hexagones réguliers. Chaque germe est ensuite déplacé d'un vecteur aléatoire uniforme dans un disque de rayon `disorder` $\times\, a$. Le réseau est exprimé dans le repère de la fissure $(\mathbf{t}, \mathbf{n})$ (rangées parallèles à la fissure) et décalé pour qu'un sommet du réseau tombe sur la pointe. Le long de la fissure, les rangées sont placées symétriquement de part et d'autre de la ligne (section 5.6) ; ailleurs, elles alternent normalement.

5.2 Régularisation de Lloyd

À chaque itération, le pavage de Voronoï est calculé et chaque germe est déplacé au centroïde de sa cellule :

$$\mathbf{x}_i \leftarrow \frac{1}{|\Omega_i|} \int_{\Omega_i} \mathbf{x}\, \mathrm{d}A.$$

Les itérations tendent vers un pavage de Voronoï centroïdal, aux cellules d'aires voisines et majoritairement hexagonales. La symétrie autour de la fissure est rétablie après chaque itération.

5.3 Pavage de Voronoï par découpe de demi-plans

La cellule du germe $\mathbf{p}$ est l'intersection du rectangle et des demi-plans $\{\mathbf{x} : (\mathbf{x}-\mathbf{m})\cdot(\mathbf{q}-\mathbf{p}) \le 0\}$, $\mathbf{m} = (\mathbf{p}+\mathbf{q})/2$, pour tous les autres germes $\mathbf{q}$. Le polygone convexe est découpé successivement (algorithme de Sutherland–Hodgman) par les germes des anneaux de grille de rang $k = 0, 1, 2, \ldots$ autour de $\mathbf{p}$ (pas de grille $h = s$). Après l'anneau k, les germes restants sont à plus de kh ; un germe à la distance D ne découpe la cellule que si $D < 2R$, où R est la distance de $\mathbf{p}$ au sommet le plus éloigné : on s'arrête dès que $kh \ge 2R$. Le coût est $O(N)$ pour une densité uniforme.

5.4 Sommets partagés et types de sommets

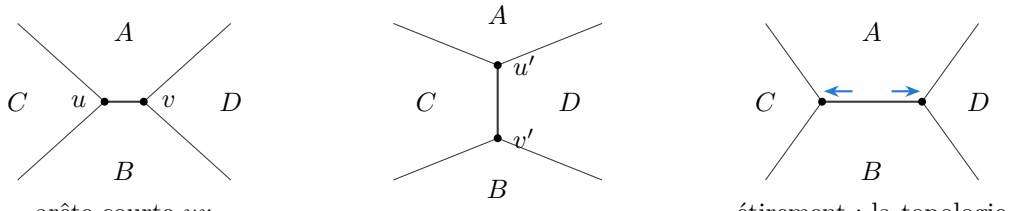
Chaque cellule est calculée indépendamment ; les sommets communs sont identifiés à $10^{-8}s$ près (table de hachage spatiale), ce qui donne une topologie cohérente : chaque arête est partagée par deux cellules (ou bordée par le domaine). Chaque sommet reçoit un type : bord gauche, droit, bas, haut, ligne de fissure, ou une combinaison (coin, embouchure de la fissure).

5.5 Arêtes courtes et jonctions à 3 branches

Pourquoi 3 branches. Dans un pavage de Voronoï générique, chaque sommet est partagé par 3 cellules : 3 parois s'y rejoignent. Une jonction de 4 parois (ou plus) est mécaniquement indésirable : deux cellules opposées ne s'y touchent qu'en un point, et la jonction en croix offre aux parois un mécanisme de glissement que n'a pas la jonction en Y. `nodegen` garantit donc que tous les sommets ont 3 branches (bords compris : 2 côtés de bord et une paroi), à la seule exception des extrémités de la pré-fissure (section 5.7). En conséquence, une arête courte n'est *jamais* fusionnée (la fusion de ses deux sommets crée un sommet à 4 branches).

Seuil. Une arête est courte si elle mesure moins de $\ell_c = \texttt{minEdgeFactor} \times \ell_{\text{ref}}$, avec $\ell_{\text{ref}} = \sqrt{L_x L_y / N}$ la longueur caractéristique des cellules : c'est le côté du carré de même aire qu'une cellule, N étant le nombre de cellules du pavage. ℓ_{ref} ne dépend que de la densité de cellules. La longueur moyenne des arêtes $\bar{\ell}$ (environ $0.62\,\ell_{\text{ref}}$ pour des hexagones, $\sqrt{2/(3\sqrt{3})}$) dépend aussi de la forme des cellules, donc du traitement lui-même, et ne convient pas comme référence. L'ancien mot-clé `minEdgeRatio` (fraction de $\bar{\ell}$) est refusé, avec la valeur équivalente de `minEdgeFactor` (environ $0.62\times$ `minEdgeRatio`).

Les arêtes courtes sont traitées de la plus courte à la plus longue (figure 7).



arête courte uv — T1 : A et B perdent un côté, C et D en gagnent un — étirement : la topologie est conservée

FIGURE 7 – Traitement d'une arête courte uv bordée par les cellules A et B ; C et D touchent chacune une extrémité. Dans les deux cas, tous les sommets gardent 3 branches.

Transition T1. L'arête uv est remplacée par une arête $u'v'$ de même milieu, de longueur `t1Margin` $\times \ell_c$ (défaut 1.2), orientée transversalement : elle est partagée par C et D, qui deviennent voisines, tandis que A et B ne se touchent plus et perdent un côté. En pratique, uv est contractée en un sommet provisoire à 4 branches, aussitôt éclaté dans l'autre sens. La transition n'est faite que si les deux sommets sont intérieurs (hors bords et fissure), si A et B gardent au moins `minSides` côtés, si les quatre cellules restent convexes, et si elle réduit le nombre d'arêtes courtes de ces cellules sans raccourcir leur plus petite arête. Elle est annulée sinon.

Étirement. Si la transition T1 est refusée ou impossible, les deux sommets sont écartés le long de l'arête, $\mathbf{u} \leftarrow \mathbf{u} - \frac{\delta}{2}\mathbf{e}$, $\mathbf{v} \leftarrow \mathbf{v} + \frac{\delta}{2}\mathbf{e}$, avec $\mathbf{e}$ la direction de l'arête et $\delta = 1.02\,\ell_c - \ell$ (la petite marge évite que deux arêtes courtes voisines se raccourcissent l'une l'autre en alternance). Un sommet contraint ne se déplace que le long de sa ligne (projection), un coin ne bouge pas (l'autre sommet fait alors tout le chemin). Le déplacement est annulé s'il rendait une cellule voisine non convexe.

Amas d'arêtes courtes. Deux arêtes courtes qui se touchent peuvent se bloquer mutuellement : aucune transition T1 isolée n'améliore la situation, et l'étirement de l'une raccourcit l'autre. En dernier recours, l'amas (arêtes courtes connexes entre sommets intérieurs, ainsi que les arêtes voisines de moins de $1.1\,\ell_c$) est contracté en un sommet, aussitôt éclaté en sommets à 3 branches. Plusieurs éclatements aléatoires sont essayés, et le premier qui réduit le nombre d'arêtes courtes sans faire passer une cellule sous `minSides` côtés est retenu.

Éclatement des sommets à 4 branches. Le pavage de Voronoï contient aussi des sommets à 4 branches ou plus : sommets quasi confondus (germes presque cocycliques), et surtout les sommets de la ligne de symétrie au-delà de la pointe (section 5.7). Un sommet v partagé par les cellules $c_0, \ldots, c_{k-1}$ (dans l'ordre angulaire) est éclaté en v, qui garde c_i, c_{i+1}, c_{i+2}, et un nouveau sommet w, qui reçoit les autres cellules. c_i et c_{i+2} deviennent voisines par l'arête vw, et w est éclaté à son tour s'il a encore plus de 3 branches. Le choix de i privilégie les cellules qui ont le moins de côtés (avec un tirage pour départager les égalités) ; un éclatement qui rendrait une cellule non convexe est refusé. Si nécessaire, l'arête créée est plus courte que le seuil, et l'étirement l'allonge ensuite.

Enchaînement et contrôle. Transitions T1, étirement, contraction des amas et éclatement des sommets restants alternent tant que le nombre de défauts (arêtes courtes, sommets à 4 branches) diminue. `nodegen` compte ensuite les branches de chaque sommet et s'arrête avec le code d'erreur 2 s'il reste un sommet à 4 branches hors des extrémités de la fissure, ou une arête sous le seuil. Les fichiers sont tout de même écrits pour pouvoir être examinés. Sur la géométrie du tutoriel, avec 20 graines par réglage, aucun échec jusqu'à `minEdgeFactor 0.22` ; à `minEdgeFactor 0.33`, 3 graines sur 20 échouent avec `disorder 0.6` ou `lattice random` (aucune avec `disorder 0.3`). Changer de graine suffit en général.

Pourquoi `minSides`. Une transition T1 retire un côté à deux cellules. Sans limite, des quadrilatères apparaissent ; avec `minSides 5`, les transitions qui en créeraient sont remplacées par des étirements. `minSides 6` réduit encore le nombre de transitions (davantage d'hexagones, moins de pentagones et d'heptagones), au prix de plus d'étirements.

5.6 Pré-fissure par symétrie

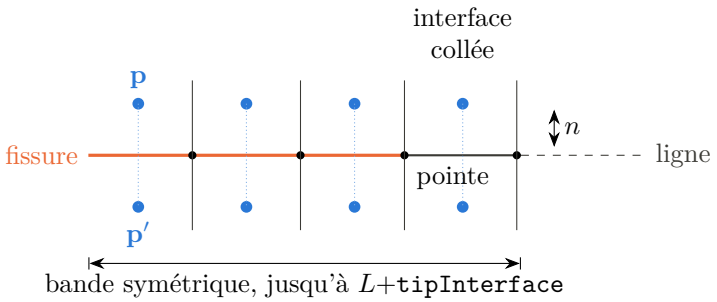


FIGURE 8 – Germes symétriques par rapport à la ligne de fissure : la médiatrice de $\mathbf{p}$ et de son symétrique $\mathbf{p}'$ est la ligne elle-même, qui est donc formée d'arêtes du pavage. La fissure est ouverte jusqu'à la pointe ; au-delà, la ligne se prolonge par une interface collée.

Dans une bande de demi-largeur $2s$ autour de la fissure, les germes du côté $n < 0$ sont supprimés et ceux du côté $n \geq 0$ sont dupliqués par symétrie. Un point $\mathbf{x}$ de la ligne est équidistant d'un germe et de son symétrique ; si les germes les plus proches de $\mathbf{x}$ sont tous dans la bande, l'ensemble des germes les plus proches est symétrique, et $\mathbf{x}$ est sur le bord de deux cellules symétriques. La ligne est donc exactement une suite d'arêtes du pavage : la fissure est rectiligne,

aucune cellule n'est coupée ni retirée. Les germes trop proches de la ligne ($n < r_{\min}/2$) sont repoussés à $n = r_{\min}/2$ pour éviter des cellules très aplaties.

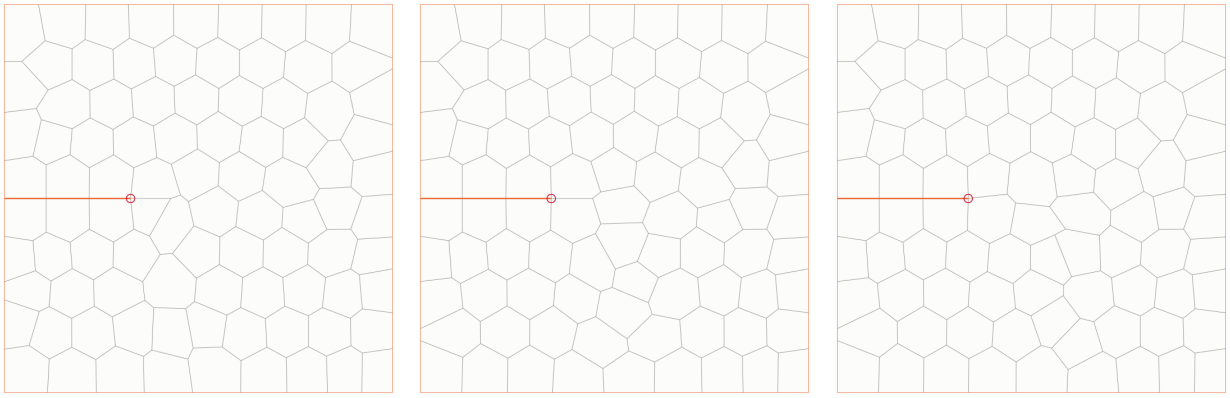
Ouverture. Le long de la fissure, les parois sont décalées de $w/2 + o/2$ au lieu de $w/2$ ($w =$ `barWidth`, $o =$ `opening`) : l'écart entre les lèvres vaut $w + o > w+$`distGlue`, donc aucun lien cohésif n'y est créé (figure 3a).

5.7 Pointe de fissure

La bande symétrique est prolongée au-delà de la pointe : la ligne se continue par une interface *collée* entre deux cellules, et la pointe est un sommet où se rejoignent quatre cellules, en face d'une interface plutôt que contre une cellule (figure 3b). C'est la seule jonction à 4 branches conservée ; il en va de même pour l'origine d'une fissure intérieure. La pointe est le sommet de la ligne le plus proche de (x_1, y_1) ; sa position effective est affichée.

Par symétrie, chaque sommet de la ligne a 4 branches (deux cellules de chaque côté). Le long de la fissure ouverte, ce n'est pas une jonction au sens mécanique : les lèvres sont libres, et chacune ne porte qu'une jonction à 3 branches. Au-delà de la pointe, en revanche, ces sommets sont éclatés (section 5.5) : chacun donne deux sommets à 3 branches qui rendent voisines deux cellules en diagonale. L'interface collée qui prolonge la fissure n'est donc pas rectiligne mais en zigzag.

Une ligne droite d'arêtes n'existe pas dans un pavage hexagonal : la rangée de pentagones qui borde la ligne doit se raccorder au réseau régulier, ce qui crée un défaut (quelques pentagones et heptagones) au bout de l'interface. `tipInterface` fixe l'étendue de la symétrie au-delà de la pointe, donc la distance entre la pointe et ce défaut (figure 9). Comme l'interface part en zigzag, sa partie rectiligne se réduit à la première arête après la pointe. Au-delà, le réseau est régulier, afin de ne pas prolonger la fissure par une interface rectiligne qui serait un chemin de rupture privilégié.



(a) `tipInterface 0` (b) `tipInterface 1` (c) `tipInterface 2`

FIGURE 9 – Étendue de la symétrie au-delà de la pointe (fissure en orange, pointe cerclée de rouge). L'interface collée qui prolonge la fissure part en zigzag ; `tipInterface` déplace le défaut de raccordement au réseau.

5.8 Décalage des parois

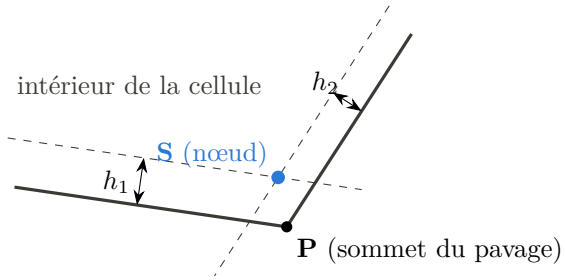


FIGURE 10 – Le nœud **S** est l'intersection des deux côtés adjacents décalés vers l'intérieur de h_1 et h_2 ($w/2$, ou $w/2 + o/2$ le long de la fissure).

Chaque cellule (polygone convexe orienté dans le sens direct) est rétrécie en décalant chacun de ses côtés vers l'intérieur : de $h = w/2$ en général, de $h = w/2 + o/2$ pour un côté de la fissure. Pour un sommet **P** entre deux côtés de normales intérieures unitaires $\mathbf{n}_1$, $\mathbf{n}_2$, le nœud **S** vérifie

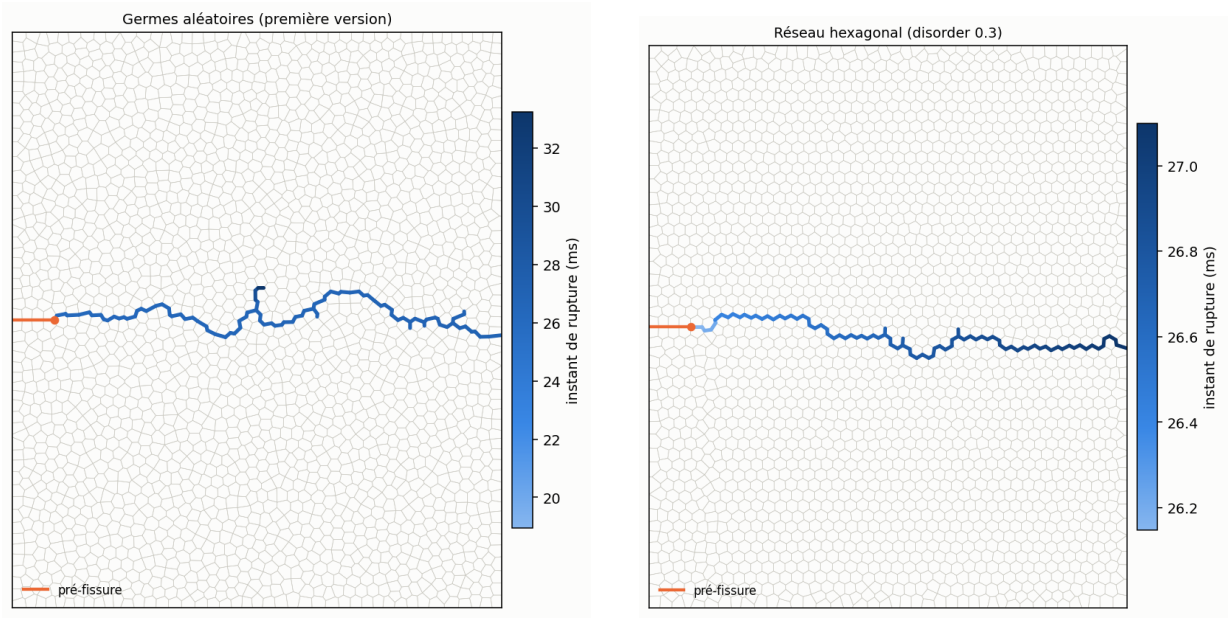
$$\mathbf{n}_1 \cdot \mathbf{S} = \mathbf{n}_1 \cdot \mathbf{P} + h_1, \qquad \mathbf{n}_2 \cdot \mathbf{S} = \mathbf{n}_2 \cdot \mathbf{P} + h_2,$$

système 2×2 résolu directement (si les deux côtés sont alignés, $\mathbf{S} = \mathbf{P} + \min(h_1, h_2)\,\mathbf{n}_1$). C'est la construction de `cellPrepro`. Deux cellules voisines décalent la *même* droite (le côté commun) de part et d'autre : leurs parois sont exactement à la distance w, condition de création des liens cohésifs par `glue` / `distGcGlue`. On vérifie enfin qu'aucun côté n'a été retourné par le décalage.

6 Validation sur un essai d'ouverture

Le nodeFile du tutoriel a été utilisé à la place de celui de `Frank-ouverture-fissure`, avec le même fichier d'input (sans `cleanShortBars`), ainsi qu'un nodeFile produit par une première version de `nodegen` (germes aléatoires, fusion seule, fissure butant sur une cellule).

Ces calculs ont été faits avec la version précédente de ***`nodegen`****, qui fusionnait les arêtes courtes et créait donc des jonctions à 4 branches ; ils n'ont pas été refaits avec les transitions T1. Les maillages actuels ne sont pas identiques à ceux de cette section.*



(a) Germes aléatoires (première version) (b) Réseau hexagonal (`tutorial_params.txt`)

FIGURE 11 – Chemins de fissuration obtenus avec L-HYPHEN (liens rompus, colorés selon l'instant de rupture ; en orange, la pré-fissure).

	Frank (`cellPrepro`)	`nodegen` aléatoire	`nodegen` hexagonal
force au pic (mN)	−44,6	−44,2	−41,8
ouverture au pic (µm)	17,6	15,7	15,6
première rupture (ms)	29,33	18,94 (isolée), puis 26,21	26,15
fin de la propagation (ms)	36,59	33,25	27,10
ruptures	141	94	86
énergie rompue (µJ)	177	119	109
longueur rompue (mm)	25,9	17,4	16,3

La référence « Frank » est le calcul sur le nodeFile de `cellPrepro` au pas $\Delta t = 1{,}75\times10^{-8}$ s ; les deux maillages `nodegen` sont calculés à $\Delta t = 5\times10^{-8}$ s sans aucun nettoyage, avec l'amortissement de flexion $\alpha_b = 0.1$. Les maillages et les pré-fissures n'étant pas identiques, seule une comparaison qualitative a du sens :

— avec les deux maillages `nodegen`, la rupture s'amorce à la pointe de la pré-fissure ; avec le réseau hexagonal, les premiers liens rompus sont ceux de l'interface qui prolonge la fissure (à 0,16 mm puis 0,36 mm de la pointe) ;
— la fissure traverse l'échantillon en un seul chemin, sans bifurcation ; elle est sinueuse avec les germes aléatoires, mais suit en zigzag une rangée du réseau hexagonal et traverse l'échantillon en moins de 1 ms : c'est l'anisotropie discutée à la section suivante ;
— le chemin plus direct explique l'énergie et la longueur rompues plus faibles qu'avec le maillage de Frank, dont la fissure bifurque.

7 Limites et améliorations possibles

7.1 Anisotropie du réseau hexagonal

Avec `lattice hex`, les rangées du réseau sont parallèles à la fissure : la fissure se propage en zigzag le long d'une rangée (figure 11b), ce qui est un artefact de l'ordre du réseau. Pistes :

— augmenter `disorder` (0.4–0.5), au prix de davantage de pentagones et d'heptagones ;
— orienter le réseau hors de la zone de la fissure selon un angle différent (par exemple 30°, ou un angle aléatoire par « grain »), les raccords formant des joints de grains ;
— partir de germes aléatoires (isotropes) et contrôler le nombre de côtés par une optimisation topologique (paragraphe suivant) plutôt que par un réseau.

7.2 Contrôle du nombre de côtés

Le nombre de côtés n'est contrôlé qu'indirectement (réseau, `minSides`). Les transitions T1 servent aujourd'hui seulement à éliminer les arêtes courtes ; une optimisation explicite est possible : transitions T1 supplémentaires, choisies pour rapprocher chaque cellule de 6 côtés (en appariant pentagones et heptagones), ou pavage de puissance (Voronoï pondéré), dont les poids permettent d'agrandir les cellules à trop peu de côtés et de réduire les autres (loi de Lewis : l'aire d'une cellule croît avec son nombre de côtés).

7.3 Géométrie

— **Bords.** Les cellules de bord sont coupées par le rectangle (d'où les quadrilatères). Placer des germes symétriques par rapport aux bords donnerait des cellules de bord complètes et des bords rectilignes formés d'arêtes du pavage, comme pour la fissure.
— **Position exacte de la pointe.** La pointe est le sommet le plus proche du point demandé (à moins d'une demi-cellule) ; on pourrait ajuster le pas du réseau le long de la fissure pour qu'un sommet tombe exactement sur (x_1, y_1).
— **Fissures multiples ou courbes, entailles en V** : symétrie par rapport à une ligne brisée ou à un arc, plusieurs bandes de symétrie (à condition qu'elles ne se chevauchent pas).
— **Domaines non rectangulaires, trous, inclusions** : découpe des cellules par un polygone quelconque au lieu du rectangle.
— **Distribution de tailles** : gradient de taille ou population bimodale, par une densité dans l'itération de Lloyd (centroïdes pondérés) ou par un pavage de puissance.

7.4 Robustesse et outillage

— Les traitements sont locaux : avec un seuil élevé (`minEdgeFactor` $\gtrsim 0.25$) et un pavage désordonné, une ou deux arêtes restent parfois trop courtes, et `nodegen` s'arrête en erreur. Une relaxation globale des sommets (réseau de ressorts avec longueur minimale) serait plus robuste.
— Proposer d'autres chargements que la traction par mors (cisaillement, flexion trois points) ; reconnaître dans le modèle d'autres dispositions de boîtes de contrôle que deux mors haut et bas.
— Un visualiseur interactif (GLFW, comme `see2`) pour naviguer dans les grands maillages.
— Des tests automatiques : topologie (chaque arête intérieure partagée par exactement deux cellules), écartement des parois, rectitude de la fissure, reproductibilité à graine fixée.

Régénérer les figures

Toutes les figures de maillage, ainsi que le bilan affiché et les lignes d'input reproduits dans ce document, sont produits par `doc/make_figures.sh` (nécessite `inkscape` et ImageMagick) à partir de `doc/tutorial_params.txt` ; les fichiers intermédiaires sont dans `doc/work/`. Les chemins de fissuration proviennent de calculs L-HYPHEN et les valeurs de la section de validation de `doc/validation_results.tex`.